CS 182 - HW 0 - Jie Wen

Sorry, I only realized the assignment required “handwritten work” after finishing it in Obsidian Markdown... By then it was already 10:30 PM on September 5th, and there was absolutely no time to rewrite everything by hand... But I promise all the content is entirely my own independent work!

1. Reflection on your learning goals at the start of the semester

(a) Initial Learning Goals

Coming into this course, my primary learning goals were shaped by my background in financial engineering and my practical experience as a quantitative researcher. My work has focused on applying machine learning models to financial time-series data to identify alpha and design trading strategies. Therefore, my initial goal for COMPSCI C182 was primarily instrumental: I wanted to master more advanced neural network architectures, such as Recurrent Neural Networks (RNNs), LSTMs, and perhaps Transformers, to improve my ability to model the complex, non-linear dynamics of financial markets. I viewed deep learning as the next powerful tool to add to my quantitative toolkit, hoping to gain a competitive edge in building more sophisticated and profitable models for forecasting, arbitrage, and portfolio management.

(b) Modification of Goals After Interacting with an LLM

My conversation with a large language model (in my case, Google's Gemini) broadened my perspective. While I prompted it about applications in finance, it naturally expanded the discussion to the foundational concepts and diverse capabilities of deep learning across various domains, such as natural language processing (NLP) for sentiment analysis on financial news. This interaction shifted my goals from being purely application-driven to also include a deeper conceptual understanding. I realized that to truly innovate, I couldn't just treat these models as black boxes. My modified goal is now not only to learn *how* to implement these architectures but to understand *why* they work—to grasp the underlying principles of representation learning, optimization landscapes, and the inductive biases of different models. This deeper knowledge will be crucial for adapting these technologies to novel problems in finance and beyond.

(c) The Role of this Class After Reading Donoho's Paper

David Donoho's paper, "Data Science at the Singularity," provides a framework for understanding the current moment in AI and, consequently, the role of this course. The prompt describes this course as taking an "intellectually conservative point of view" focused on mathematical intuition and tradition, which at first might seem at odds with the rapid, breakthrough-driven world of modern deep learning. However, Donoho’s argument that progress is driven by a culture of "frictionless reproducibility, FR" (shared data, code, and competitive challenges) actually highlights the critical importance of this course's approach.

The FR paradigm excels at empirical advancement and scaling existing ideas, but it doesn't necessarily produce the foundational knowledge required to understand *why* certain architectures are so effective or what their fundamental limitations are. This class, therefore, is not just about learning the current state-of-the-art; it's about learning the "first principles." It provides the mathematical and theoretical bedrock upon which the entire empirical enterprise described by Donoho is built. For me, this course is an opportunity to move beyond being a mere participant in the FR ecosystem—someone who can download a model and apply it—and become someone who can reason critically about these systems. In the world of quantitative finance, where misinterpreting a model's limitations can lead to catastrophic losses, this deep, foundational understanding is invaluable for robust and reliable innovation.

(d) Complementary Activities and Guidance

My understanding of deep learning can be enhanced by activities that bridge the theoretical foundations of this course with the fast-paced, practical world Donoho describes.

- **Complementary Activities:**
  - **Applied Projects:** I plan to actively apply the concepts learned in class to my own domain of interest, quantitative finance. For example, I could use the theoretical knowledge of attention mechanisms to build and rigorously test a Transformer-based model for predicting asset volatility.
  - **Kaggle Competitions:** Participating in competitions would allow me to engage directly in the "competitive challenge" aspect of Donoho's FR framework, testing my skills against others on standardized datasets and learning from the practical techniques shared by the community.
  - **Reading Groups:** Joining or forming a reading group to discuss recent, high-impact papers from conferences like NeurIPS or ICML would help me stay abreast of the latest advancements that a structured curriculum might not cover in real-time.
- **Helpful Guidance:**
  - **From Peers:** I would benefit from collaborating with peers from different backgrounds, especially those with stronger software engineering or pure computer science expertise. Their perspectives would complement my quantitative focus and would be invaluable in team-based projects.
  - **From Course Staff:** I would find it helpful to get guidance from the professor and TAs on how to connect the course's theoretical concepts to current research frontiers. For instance, during office hours, I would appreciate discussions on how a specific mathematical proof or model property discussed in a lecture informs the architecture of a state-of-the-art model like those used in my internships. This would help close the gap between theory and practice.

---

2. Vector Calculus Review

(a) $\frac{d}{dx}(x^T c)$

Type: $x^T c$ is a **scalar**, so $\frac{d}{dx}(x^T c)$ is a **row vector** of size $1 \times n$.
Write $f(x) = x^T c = \sum_{i=1}^{n} x_i c_i$. Then:

$$\frac{\partial f}{\partial x_j} = c_j$$

So the derivative is:

$$\frac{d}{dx}(x^T c) = [c_1, c_2, \ldots, c_n] = \boxed{c^T}$$

(b) $\frac{d}{dx}\|x\|_2^2$

Type: $\|x\|_2^2 = x^T x$ is a **scalar**, so the derivative is a **row vector** $1 \times n$.
$\|x\|_2^2 = x^T x = \sum_{i=1}^n x_i^2$. For an arbitrary entry:

$$\frac{\partial}{\partial x_j}\|x\|_2^2 = 2x_j$$

So the derivative is:

$$\frac{d}{dx}\|x\|_2^2 = [2x_1, 2x_2, \ldots, 2x_n] = \boxed{2x^T}$$

(c) $\frac{d}{dx}(Ax)$

Type: Ax is a **vector** in $\mathbb{R}^n$, so the derivative is an $n \times n$ **matrix**.
Let $f(x) = Ax$, where $A \in \mathbb{R}^{n \times n}$. Then:

$$f_i(x) = \sum_{k=1}^n A_{ik}x_k \quad \Rightarrow \quad \frac{\partial f_i}{\partial x_j} = A_{ij}$$

So the derivative matrix is:

$$\frac{d}{dx}(Ax) = \boxed{A}$$

(d) $\frac{d}{dx}(x^T Ax)$

Type: $x^T Ax$ is a **scalar**, so the derivative is a **row vector** $1 \times n$.
Let $f(x) = x^T Ax = \sum_{i,j} A_{ij}x_i x_j$. Then:

$$\frac{\partial f}{\partial x_k} = \sum_j A_{kj}x_j + \sum_i A_{ik}x_i = (Ax)_k + (A^T x)_k$$

So the derivative row vector is:

$$\frac{d}{dx}(x^T Ax) = (Ax + A^T x)^T = x^T A^T + x^T A = \boxed{x^T(A + A^T)}$$

(e) Condition for $\frac{d}{dx}(x^T Ax) = 2x^T A$

From part (d):

$$\frac{d}{dx}(x^T Ax) = x^T(A + A^T)$$

We want this to equal $2x^T A$, i.e.,

$$x^T(A + A^T) = 2x^T A \quad \Rightarrow \quad x^T(A^T - A) = 0$$

This holds for all x if and only if $A^T - A = 0$, i.e., A is symmetric:

$$\boxed{A = A^T}$$

---

3. Least Squares and the Min-norm problem from the Perspective of SVD

(a) Overdetermined case ($m > n$), least squares $\min_w \|Xw - y\|_2^2$

Assume X has full **column** rank n. The normal equations give

$$X^T Xw = X^T y.$$

Because $X^T X$ is invertible (positive definite when full column rank),

$$\boxed{w^* = (X^T X)^{-1} X^T y}$$

(b) Plug in SVD and simplify

Using $X = U\Sigma V^T$:

$$X^TX = V\Sigma^TU^TU\Sigma V^T = V(\Sigma^T\Sigma)V^T,$$

and

$$(X^TX)^{-1} = V(\Sigma^T\Sigma)^{-1}V^T.$$

Also $X^T = V\Sigma^TU^T$. Thus

$$w^* = (X^TX)^{-1}X^Ty = V(\Sigma^T\Sigma)^{-1}V^TV\Sigma^TU^Ty = V(\Sigma^T\Sigma)^{-1}\Sigma^TU^Ty.$$

Because $(\Sigma^T\Sigma)^{-1}\Sigma^T = \Sigma^\dagger$ (for full column rank),

$$\boxed{w^* = V\Sigma^\dagger U^Ty = X^\dagger y}$$

(Here $\Sigma^\dagger$ is $n \times m$, so the product dimensions match.)

(c) The left-inverse property

Write $A = (X^TX)^{-1}X^T = X^\dagger$. Then

$$AX = (X^TX)^{-1}X^TX = I_n.$$

So A is a **left-inverse** of X (an $n \times m$ matrix satisfying $AX = I_n$). Using SVD,

$$A = V\Sigma^\dagger U^T, \quad \boxed{AX = V\Sigma^\dagger U^TU\Sigma V^T = V(\Sigma^\dagger\Sigma)V^T = I_n}$$

(Here $\Sigma^\dagger\Sigma = I_n$ because Σ has full column rank.)

(d) Underdetermined case ($m < n$), minimum-norm solution

Assume X has full **row** rank m. Among all w satisfying $Xw = y$ there are infinitely many; the minimum Euclidean norm solution is found by Lagrange multipliers:

Minimize $\frac{1}{2}\|w\|_2^2$ subject to $Xw = y$. Stationarity gives

$$w = X^T\lambda$$

and feasibility $XX^T\lambda = y$. Hence $\lambda = (XX^T)^{-1}y$ and

$$\boxed{w^* = X^T(XX^T)^{-1}y}$$

(e) Plug in SVD and simplify

Using $X = U\Sigma V^T$,

$$XX^T = U\Sigma V^TV\Sigma^TU^T = U(\Sigma\Sigma^T)U^T,$$

so $(XX^T)^{-1} = U(\Sigma\Sigma^T)^{-1}U^T$. Then

$$w^* = X^T(XX^T)^{-1}y = V\Sigma^TU^TU(\Sigma\Sigma^T)^{-1}U^Ty = V(\Sigma^T(\Sigma\Sigma^T)^{-1})U^Ty.$$

But $\Sigma^T(\Sigma\Sigma^T)^{-1} = \Sigma^\dagger$ (for full row rank), so

$$\boxed{w^* = V\Sigma^\dagger U^Ty = X^\dagger y}$$

So again the pseudoinverse $X^\dagger$ gives the min-norm solution.

(f) The right-inverse property

Set $B = X^T(XX^T)^{-1} = X^\dagger$ (now B is $n \times m$). Then

$$XB = XX^T(XX^T)^{-1} = I_m,$$

so B is a **right-inverse** of X. Using SVD,

$$\boxed{XB = U\Sigma V^TV\Sigma^\dagger U^T = U(\Sigma\Sigma^\dagger)U^T = I_m}$$

because $\Sigma\Sigma^\dagger = I_m$ when Σ has full row rank.

---

4. The 5 Interpretations of Ridge Regression

(a) Optimization / vector-calculus

Objective

$$f(w) = \|y - Xw\|_2^2 + \lambda\|w\|_2^2 = (y - Xw)^T(y - Xw) + \lambda w^Tw.$$

Compute gradient, using $\nabla_w(y - Xw)^T(y - Xw) = -2X^T(y - Xw)$ and $\nabla_w\lambda w^Tw = 2\lambda w$:

$$\nabla_w f(w) = -2X^T(y - Xw) + 2\lambda w = -2X^Ty + 2X^TXw + 2\lambda w.$$

Set $\nabla_w f(w) = 0$:

$$X^TXw + \lambda w = X^Ty \implies (X^TX + \lambda I)w = X^Ty.$$

Thus the minimizer is

$$\boxed{w^* = (X^T X + \lambda I)^{-1} X^T y}$$

(b) SVD / singular-value "shrinkage"

Write $X = U\Sigma V^T$ (full SVD). Then

$$X^T X = V\Sigma^T \Sigma V^T = V D V^T, \quad D = \operatorname{diag}(\sigma_i^2).$$

So

$$(X^T X + \lambda I)^{-1} = V(D + \lambda I)^{-1} V^T,$$

and

$$(X^T X + \lambda I)^{-1} X^T = V(D + \lambda I)^{-1} V^T V \Sigma^T U^T = V(D + \lambda I)^{-1} \Sigma^T U^T.$$

Because Σ^T has σ_i on its diagonal, this equals

$$\boxed{(X^T X + \lambda I)^{-1} X^T = V \operatorname{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \lambda}\right) U^T} \implies \boxed{w = V(\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T U^T y}$$

So each singular-direction is scaled by factor $\frac{\sigma_i}{\sigma_i^2 + \lambda}$.

- If $\sigma_i \ll \lambda$: $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx \frac{\sigma_i}{\lambda} \approx 0$. **Small singular values are heavily suppressed** (directions with tiny signal are shrunk toward 0).
- If $\sigma_i \gg \lambda$: $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx \frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$. For large singular values the factor approaches $1/\sigma_i$, so the corresponding direction is preserved similarly to the ordinary pseudoinverse.

(c) MAP interpretation

Model given: $Y = XW + \sqrt{\lambda} N$ with $N \sim \mathcal{N}(0, I)$. So conditional likelihood

$$p(y \mid w) \propto \exp\left(-\frac{1}{2\lambda} \|y - Xw\|_2^2\right).$$

Prior: $W \sim \mathcal{N}(0, I)$ so

$$p(w) \propto \exp\left(-\frac{1}{2} \|w\|_2^2\right).$$

MAP maximizes $\log p(w \mid y) \propto \log p(y \mid w) + \log p(w)$, i.e.

$$\arg\max_w \left[-\frac{1}{2\lambda} \|y - Xw\|_2^2 - \frac{1}{2} \|w\|_2^2\right]$$

which is equivalent (multiply by 2λ and flip sign) to minimizing

$$\|y - Xw\|_2^2 + \lambda \|w\|_2^2,$$

i.e. exactly the ridge objective (1). Hence ridge is the MAP estimator under that Gaussian prior/noise model.

(d) Fake (augmented) data — vertical augmentation

Define

$$\hat{y} = \begin{bmatrix} y \\ 0_d \end{bmatrix}, \quad \hat{X} = \begin{bmatrix} X \\ \sqrt{\lambda} I_d \end{bmatrix}.$$

Then

$$\|\hat{y} - \hat{X} w\|_2^2 = \|y - Xw\|_2^2 + \|\sqrt{\lambda} I_d w - 0_d\|_2^2 = \|y - Xw\|_2^2 + \lambda \|w\|_2^2.$$

So $\arg\min_w \|\hat{y} - \hat{X} w\|_2^2 = \arg\min_w \|y - Xw\|_2^2 + \lambda \|w\|_2^2$. The solutions coincide.

(e) Fake features — horizontal augmentation, min-norm solution

Construct $\tilde{X} = \begin{bmatrix} X & \sqrt{\lambda} I_n \end{bmatrix}$ and solve

$$\min_\eta \|\eta\|_2^2 \quad \text{s.t.} \quad \tilde{X} \eta = y,$$

write $\eta = \begin{bmatrix} w \\ v \end{bmatrix}$ with $w \in \mathbb{R}^d, v \in \mathbb{R}^n$. The constraint is

$$Xw + \sqrt{\lambda} v = y \implies v = \frac{1}{\sqrt{\lambda}} (y - Xw).$$

Objective becomes

$$\|\eta\|_2^2 = \|w\|_2^2 + \|v\|_2^2 = \|w\|_2^2 + \frac{1}{\lambda} \|y - Xw\|_2^2.$$

Minimizing $\|\eta\|_2^2$ over w is equivalent (multiply by λ) to minimizing

$$\|y - Xw\|_2^2 + \lambda \|w\|_2^2,$$

the ridge objective. Therefore the first d coordinates of the min-norm solution η^* are exactly the ridge minimizer w^*.

(f) Pseudoinverse form and equivalence

For underdetermined $\tilde{X}$ (more columns than rows), the min-norm solution is

$$\eta^* = \tilde{X}^T(\tilde{X}\tilde{X}^T)^{-1}y.$$

Compute $\tilde{X}\tilde{X}^T = XX^T + \lambda I_n$. Also

$$\tilde{X}^T = \begin{bmatrix} X^T \\ \sqrt{\lambda} I_n \end{bmatrix}$$

so

$$\eta^* = \begin{bmatrix} X^T \\ \sqrt{\lambda} I_n \end{bmatrix} (XX^T + \lambda I_n)^{-1} y = \begin{bmatrix} X^T(XX^T + \lambda I_n)^{-1}y \\ \sqrt{\lambda}(XX^T + \lambda I_n)^{-1}y \end{bmatrix}.$$

Hence the first d coordinates are

$$\boxed{\tilde{w} = X^T(XX^T + \lambda I)^{-1}y}$$

To show equivalence to $(X^TX + \lambda I)^{-1}X^Ty$, observe (one way is via SVD): using $X = U\Sigma V^T$ we obtain both expressions equal to

$$V \operatorname{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \lambda}\right) U^T y,$$

so they are identical. Equivalently, the matrix identity

$$X^T(XX^T + \lambda I)^{-1} = (X^TX + \lambda I)^{-1}X^T$$

holds (it is a specialization of the Woodbury / matrix inversion identities). Thus the two forms of $\tilde{w}$ are equal:

$$\boxed{\tilde{w} = X^T(XX^T + \lambda I)^{-1}y = (X^TX + \lambda I)^{-1}X^Ty}$$

(g) Limit $\lambda \to \infty$ (shrinkage)

Consider $\hat{w}(\lambda) = (X^TX + \lambda I)^{-1}X^Ty$. Factor λ:

$$\hat{w}(\lambda) = \frac{1}{\lambda}\left(I + \frac{1}{\lambda}X^TX\right)^{-1}X^Ty.$$

As $\lambda \to \infty$ the prefactor $1/\lambda \to 0$ and $(I + \frac{1}{\lambda}X^TX)^{-1} \to I$, so

$$\boxed{\hat{w}(\lambda) \to 0 \quad \text{as} \quad \lambda \to \infty}$$

Hence coefficients are shrunk toward zero.

(h) Limit $\lambda \to 0$

- If X is **tall** (overdetermined, $n \geq d$) and has full column rank, then X^TX is invertible. As $\lambda \to 0$,

  $$\hat{w}(\lambda) \to (X^TX)^{-1}X^Ty,$$

  the usual unique OLS solution.
- If X is **wide** (underdetermined, $d > n$) but has full row rank (rank = n), then XX^T is invertible and using the alternate form

  $$\hat{w}(\lambda) = X^T(XX^T + \lambda I)^{-1}y,$$

  we get as $\lambda \to 0$,

  $$\hat{w}(\lambda) \to X^T(XX^T)^{-1}y,$$

  which is the **minimum-norm** solution of the underdetermined least-squares problem $\min_w \|w\|_2$ s.t. $Xw = y$ (i.e. the Moore-Penrose pseudoinverse solution $X^\dagger y$).
- If X is rank-deficient (so X^TX singular) then at $\lambda = 0$ the ordinary least-squares problem may have infinitely many solutions; ridge with $\lambda > 0$ picks a unique solution and as $\lambda \downarrow 0$ it approaches the minimum-norm least-squares solution (if the limit exists), i.e. the pseudoinverse solution.

Summary: $\lambda \downarrow 0$ recovers the ordinary least-squares solution when it is unique; when OLS solutions are non-unique (wide case), the ridge limit picks the minimum-norm solution.

---

5. ReLU Elbow Update under SGD

(a) Single ReLU: formulas and derivatives

(i) Elbow location e:

The elbow occurs where the input to the ReLU is zero:

$$wx + b = 0 \quad \Rightarrow \quad x = -\frac{b}{w}$$

So,

$$\boxed{e = -\frac{b}{w}}$$

(ii) Derivative of loss w.r.t. $\phi(x)$:

$$\frac{dl}{d\phi(x)} = \phi(x) - y$$

$$\boxed{\frac{dl}{d\phi(x)} = \phi(x) - y}$$

(iii) Partial derivative w.r.t. w:

$$\frac{\partial l}{\partial w} = (\phi(x) - y) \cdot \frac{\partial \phi}{\partial w}$$

Since $\frac{\partial \phi}{\partial w} = x$ when $wx + b > 0$, and 0 otherwise:

$$\boxed{\frac{\partial l}{\partial w} = \begin{cases} (\phi(x) - y)x, & wx + b > 0 \\ 0, & \text{else} \end{cases}}$$

(iv) Partial derivative w.r.t. b:

$$\frac{\partial l}{\partial b} = (\phi(x) - y) \cdot \frac{\partial \phi}{\partial b}$$

Since $\frac{\partial \phi}{\partial b} = 1$ when $wx + b > 0$, and 0 otherwise:

$$\boxed{\frac{\partial l}{\partial b} = \begin{cases} \phi(x) - y, & wx + b > 0 \\ 0, & \text{else} \end{cases}}$$

(b) Behavior when $\phi(x) - y = 1$

Given $\phi(x) - y = 1$, perform gradient descent updates:

$$w' = w - \lambda \frac{\partial l}{\partial w}, \quad b' = b - \lambda \frac{\partial l}{\partial b}$$

(i) $\phi(x) = 0$:

- Since $wx + b \leq 0$, both gradients are zero.
- No update: slope and elbow remain unchanged.

(ii) $w > 0, x > 0, \phi(x) > 0$:

- Gradients: $\frac{\partial l}{\partial w} = x > 0$, $\frac{\partial l}{\partial b} = 1$
- Updates: $w' = w - \lambda x$, $b' = b - \lambda$
- Slope decreases, elbow moves right.

```
before:       e     x
             /       .
            /        .  (sample at x, phi(x) > y)
           /
----------/----------------
          ^
          elbow

after: slope less steep, elbow shifted toward x
```

(iii) $w > 0, x < 0, \phi(x) > 0$:

- Gradients: $\frac{\partial l}{\partial w} = x < 0$, $\frac{\partial l}{\partial b} = 1$
- Updates: $w' = w - \lambda x$ (increases), $b' = b - \lambda$
- Slope increases, elbow moves right.

```
before: x   e
        .   \
        .    \
             \
---------------\------------
               ^
               elbow
```

```
after: slope steeper, elbow moved right (toward x)
```

(iv) $w < 0, x > 0, \phi(x) > 0$:

- Gradients: $\frac{\partial l}{\partial w} = x > 0$, $\frac{\partial l}{\partial b} = 1$
- Updates: $w' = w - \lambda x$ (more negative), $b' = b - \lambda$
- Slope becomes more negative, elbow moves left.

```
before:        e     x
             \        .
              \       .
               \_____
----------------------
            ^
           elbow

after: slope more negative, elbow moved left (toward x)
```

(c) Full Network Elbow Location

For the full network $f(x) = W^{(2)}\phi(W^{(1)}x + b)$, the i-th ReLU has the form $\phi_i(x) = \max(0, W_{i,1}^{(1)}x + b_i)$.

The elbow location for the i -th ReLU is:

$$\boxed{e_i = -\frac{b_i}{W_{i,1}^{(1)}}}$$

(d) Updated Elbow Location

After one SGD update with learning rate λ, we need the gradients:

For the i-th component:

- $\frac{\partial l}{\partial W_{i,1}^{(1)}} = \frac{\partial l}{\partial f} \cdot W_{1,i}^{(2)} \cdot \mathbf{1}[\phi_i(x) > 0] \cdot x$
- $\frac{\partial l}{\partial b_i} = \frac{\partial l}{\partial f} \cdot W_{1,i}^{(2)} \cdot \mathbf{1}[\phi_i(x) > 0]$

Where $\frac{\partial l}{\partial f} = f(x) - y$.

The updates are:

- $W_{i,1}^{(1)'} = W_{i,1}^{(1)} - \lambda \frac{\partial l}{\partial W_{i,1}^{(1)}}$
- $b_i' = b_i - \lambda \frac{\partial l}{\partial b_i}$

Therefore, the new elbow location is:

$$e_i' = -\frac{b_i'}{W_{i,1}^{(1)'}} = -\frac{b_i - \lambda \frac{\partial l}{\partial b_i}}{W_{i,1}^{(1)} - \lambda \frac{\partial l}{\partial W_{i,1}^{(1)}}}$$

When $\phi_i(x) > 0$:

$$\boxed{e_i' = -\frac{b_i - \lambda(f(x) - y)W_{1,i}^{(2)}}{W_{i,1}^{(1)} - \lambda(f(x) - y)W_{1,i}^{(2)}x}}$$

This shows how the elbow positions adapt based on the prediction error $(f(x) - y)$ and the specific values of the weights and input.

---

6. Coding Fully Connected Networks (It's included at the end of this document. I've attached it.)

(I haven't found where to submit multiple files in Gradescope...)

Based on the results from training both the three-layer and five-layer networks on a small dataset of 50 examples, I observed the following key differences in training difficulty:

1. **Training Accuracy**: The three-layer network achieved 100% training accuracy within 20 epochs, while the five-layer network reached only 96% training accuracy in the same number of epochs. This indicates that the three-layer network was able to overfit the small dataset more easily and completely.
2. **Loss Convergence**: The three-layer network's loss decreased rapidly and reached very low values (close to zero), while the five-layer network's loss decreased more slowly and remained higher throughout training. The three-layer network showed a smoother and faster convergence.

3. **Optimization Challenges**: The five-layer network appeared to be more difficult to optimize, likely due to increased model complexity and depth-related issues such as vanishing gradients, which can make it harder for gradients to propagate effectively through more layers. This required more careful tuning of hyperparameters (like weight scale) to achieve good performance.

In summary, the five-layer network was comparatively more difficult to train than the three-layer network, as it did not achieve perfect overfitting on the small dataset within the same number of epochs and exhibited slower convergence. This highlights the increased optimization challenges that come with deeper network architectures.

---

7. Homework Process and Study Group

(a)

My previous course materials on optimization, machine learning and linear algebra

(b)

Nobody

(c)

5 Hours..
Sorry, I only realized the assignment required “handwritten work” after finishing it in Obsidian Markdown... By then it was already 10:30 PM on September 5th, and there was absolutely no time to rewrite everything by hand... But I promise all the content is entirely my own independent work!

Fully-Connected Neural Nets

In this notebook we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
  out = # the output

  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive derivative of loss with respect to outputs and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

```
# As usual, a bit of setup

import time
import numpy as np
import matplotlib.pyplot as plt
from deeplearning.classifiers.fc_net import *
from deeplearning.data_utils import get_CIFAR10_data
from deeplearning.gradient_check import eval_numerical_gradient,
```

```
eval_numerical_gradient_array
from deeplearning.solver import Solver

plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of
plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-
modules-in-ipython

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) +
np.abs(y))))

# Lddoad the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in data.items():
    print('%s: ' % k, v.shape)
```

```
deeplearning/datasets/cifar-10-batches-py/data_batch_1
deeplearning/datasets/cifar-10-batches-py/data_batch_2
deeplearning/datasets/cifar-10-batches-py/data_batch_3

deeplearning/datasets/cifar-10-batches-py/data_batch_4
deeplearning/datasets/cifar-10-batches-py/data_batch_5
deeplearning/datasets/cifar-10-batches-py/test_batch
X_train:  (49000, 3, 32, 32)
y_train:  (49000,)
X_val:  (1000, 3, 32, 32)
y_val:  (1000,)
X_test:  (1000, 3, 32, 32)
y_test:  (1000,)
```

Affine layer: forward

Open the file `deeplearning/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3
```

```
input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs,
*input_shape)
w = np.linspace(-0.2, 0.3,
num=weight_size).reshape(np.prod(input_shape), output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around 1e-9.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

```
# Test the affine_backward function

x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w,
b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w,
b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w,
b)[0], b, dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)
# The error should be around 1e-10
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  1.0898824908503163e-09
dw error:  1.2673684742741393e-10
db error:  8.658502807816594e-12
```

ReLU layer: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,
],
                        [ 0.,          0.,          0.04545455,
0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,
]])

# Compare your output with ours. The error should be around 1e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

ReLU layer: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking. Note that the ReLU activation is not differentiable at 0, but typically we don't worry about this and simply assign either 0 or 1 as the derivative by convention.

```
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0],
x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be around 1e-12
```

```
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756152972411477e-12
```

"Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `deeplearning/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

```
from deeplearning.layer_utils import affine_relu_forward,
affine_relu_backward

x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x:
affine_relu_forward(x, w, b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w:
affine_relu_forward(x, w, b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b:
affine_relu_forward(x, w, b)[0], b, dout)

print('Testing affine_relu_forward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward:
dx error:  8.485756833268294e-11
dw error:  7.142529459649762e-10
db error:  4.963956579814275e-11
```

Loss layers: Softmax and SVM

Here we provide two loss functions that we will use to train our deep neural networks. You should understand how they work by looking at the implementations in `deeplearning/layers.py`.

You can make sure that the implementations are correct by running the following:

```
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)

dx_num = eval_numerical_gradient(lambda x: svm_loss(x, y)[0], x,
verbose=False)
loss, dx = svm_loss(x, y)

# Test svm_loss function. Loss should be around 9 and dx error should
be 1e-9
print('Testing svm_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))

dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,
verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be 2.3 and dx error should
be 1e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing svm_loss:
loss:  9.000304146950834
dx error:  1.4021566006651672e-09

Testing softmax_loss:
loss:  2.3026159675439377
dx error:  7.507393684572627e-09
```

Two-layer network

Open the file `deeplearning/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. This class will serve as a model for the other networks you will implement in this assignment, so read through it to make sure you understand the API. Run the cell below to test your implementation.

```
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-2
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C,
weight_scale=std)

print ('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print ('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,
14.57198434, 15.33206765,  16.09215096],
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,
14.81149128, 15.49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,
15.05099822, 15.66781506,  16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print ('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time
loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization
loss'

for reg in [0.0, 0.7]:
    print ('Running numeric gradient check with reg = ', reg)
```

```
    model.reg = reg
    loss, grads = model.loss(X, y)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],
verbose=False)
        print ('%s relative error: %.2e' % (name, rel_error(grad_num,
grads[name])))
```

```
Testing initialization ...
Testing test-time forward pass ...
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.52e-08
W2 relative error: 3.21e-10
b1 relative error: 6.55e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10
```

Solver

Following a modular design, for this assignment we have split the logic for training models into a separate class from the models themselves.

Open the file `deeplearning/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves at least `50%` accuracy on the validation set.

```
model = TwoLayerNet()
solver = None

##############################################################################
########
# TODO: Use a Solver instance to train a TwoLayerNet that achieves at
least  #
# 50% accuracy on the validation set.
#
##############################################################################
########
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_dim=input_size, hidden_dim=hidden_size,
```

```
num_classes=num_classes)
solver = Solver(model, data,
                update_rule='sgd',
                optim_config={
                  'learning_rate': 1e-3,
                },
                lr_decay=0.95,
                num_epochs=10, batch_size=100,
                print_every=100)
solver.train()
################################################################################
########
#                             END OF YOUR CODE
#
################################################################################
########

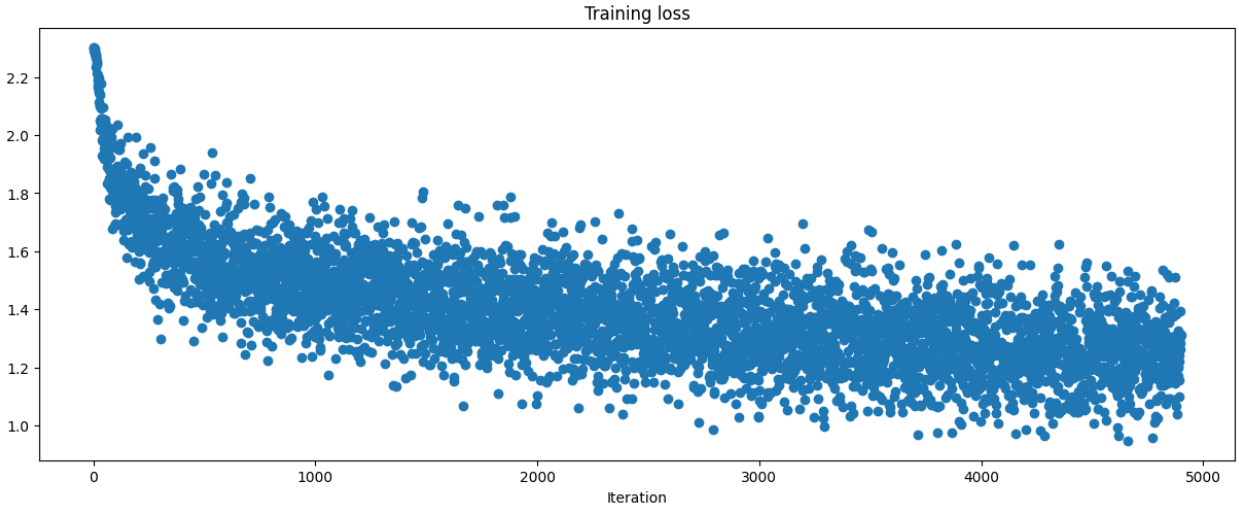
(Iteration 1 / 4900) loss: 2.302400
(Epoch 0 / 10) train acc: 0.132000; val_acc: 0.120000
(Iteration 101 / 4900) loss: 1.694286
(Iteration 201 / 4900) loss: 1.869454
(Iteration 301 / 4900) loss: 1.748515
(Iteration 401 / 4900) loss: 1.558225
(Epoch 1 / 10) train acc: 0.445000; val_acc: 0.440000
(Iteration 501 / 4900) loss: 1.621671
(Iteration 601 / 4900) loss: 1.766529
(Iteration 701 / 4900) loss: 1.534654
(Iteration 801 / 4900) loss: 1.638880
(Iteration 901 / 4900) loss: 1.649830
(Epoch 2 / 10) train acc: 0.501000; val_acc: 0.470000
(Iteration 1001 / 4900) loss: 1.650723
(Iteration 1101 / 4900) loss: 1.490884
(Iteration 1201 / 4900) loss: 1.639373
(Iteration 1301 / 4900) loss: 1.457308
(Iteration 1401 / 4900) loss: 1.329819
(Epoch 3 / 10) train acc: 0.476000; val_acc: 0.476000
(Iteration 1501 / 4900) loss: 1.343836
(Iteration 1601 / 4900) loss: 1.507205
(Iteration 1701 / 4900) loss: 1.435681
(Iteration 1801 / 4900) loss: 1.239882
(Iteration 1901 / 4900) loss: 1.441253
(Epoch 4 / 10) train acc: 0.536000; val_acc: 0.488000
(Iteration 2001 / 4900) loss: 1.351129
(Iteration 2101 / 4900) loss: 1.386864
(Iteration 2201 / 4900) loss: 1.531870
(Iteration 2301 / 4900) loss: 1.204053
(Iteration 2401 / 4900) loss: 1.317452
(Epoch 5 / 10) train acc: 0.528000; val_acc: 0.475000
(Iteration 2501 / 4900) loss: 1.367684
(Iteration 2601 / 4900) loss: 1.290212
```

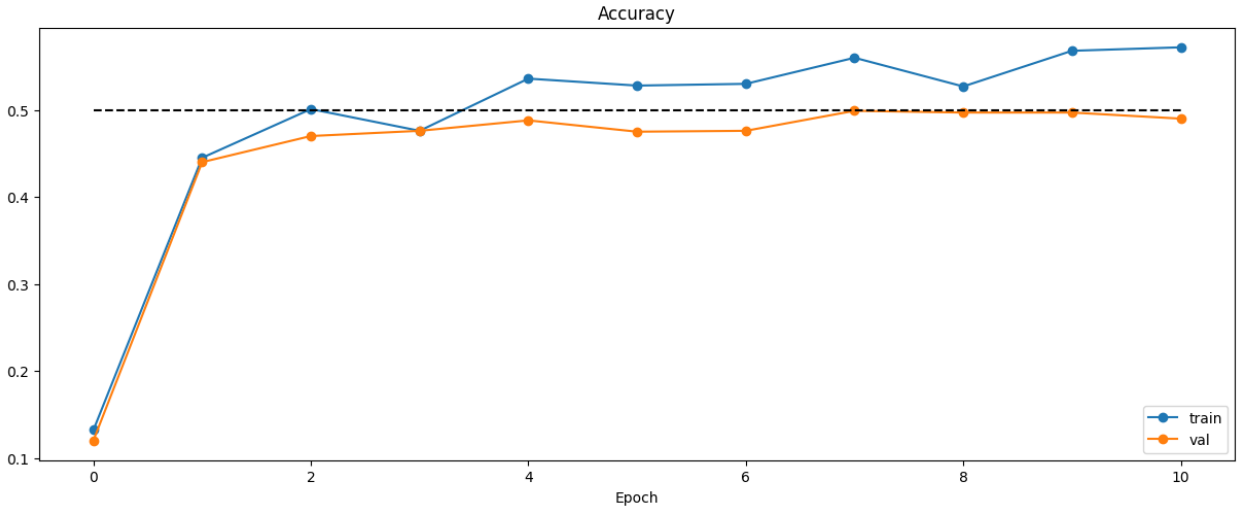
```
(Iteration 2701 / 4900) loss: 1.497268
(Iteration 2801 / 4900) loss: 1.517959
(Iteration 2901 / 4900) loss: 1.386905
(Epoch 6 / 10) train acc: 0.530000; val_acc: 0.476000
(Iteration 3001 / 4900) loss: 1.219006
(Iteration 3101 / 4900) loss: 1.385023
(Iteration 3201 / 4900) loss: 1.370166
(Iteration 3301 / 4900) loss: 1.271639
(Iteration 3401 / 4900) loss: 1.380382
(Epoch 7 / 10) train acc: 0.560000; val_acc: 0.499000
(Iteration 3501 / 4900) loss: 1.336242
(Iteration 3601 / 4900) loss: 1.431402
(Iteration 3701 / 4900) loss: 1.113991
(Iteration 3801 / 4900) loss: 1.295434
(Iteration 3901 / 4900) loss: 1.192896
(Epoch 8 / 10) train acc: 0.527000; val_acc: 0.497000
(Iteration 4001 / 4900) loss: 1.336841
(Iteration 4101 / 4900) loss: 1.220456
(Iteration 4201 / 4900) loss: 1.235379
(Iteration 4301 / 4900) loss: 1.301909
(Iteration 4401 / 4900) loss: 1.427697
(Epoch 9 / 10) train acc: 0.568000; val_acc: 0.497000
(Iteration 4501 / 4900) loss: 1.316022
(Iteration 4601 / 4900) loss: 1.128597
(Iteration 4701 / 4900) loss: 1.034213
(Iteration 4801 / 4900) loss: 1.264287
(Epoch 10 / 10) train acc: 0.572000; val_acc: 0.490000
```

```
# Run this cell to visualize training loss and train / val accuracy,
and save the log file of the
# experiment for submission.

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```





Multilayer network

Next you will implement a fully-connected network with an arbitrary number of hidden layers.

Read through the `FullyConnectedNet` class in the file `deeplearning/classifiers/fc_net.py`.

Implement the initialization, the forward pass, and the backward pass. For the moment don't worry about implementing dropout or batch normalization; we will add those features soon.

Initial loss and gradient check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. Do the initial losses seem reasonable?

For gradient checking, you should expect to see errors around 1e-6 or less.

```
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print ('Running check with reg = ', reg)
    model = FullyConnectedNet([H1, H2], input_dim=D, num_classes=C,
                              reg=reg, weight_scale=5e-2,
dtype=np.float64)

    loss, grads = model.loss(X, y)
    print ('Initial loss: ', loss)

    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],
verbose=False, h=1e-5)
        print ('%s relative error: %.2e' % (name, rel_error(grad_num,
grads[name])))
```

```
Running check with reg =  0
Initial loss:  2.3041588167934477
W1 relative error: 9.67e-08
W2 relative error: 4.49e-07
W3 relative error: 2.16e-06
b1 relative error: 1.94e-08
b2 relative error: 4.42e-09
b3 relative error: 9.56e-11
Running check with reg =  3.14
Initial loss:  7.308933593905188
W1 relative error: 8.33e-09
W2 relative error: 1.86e-07
W3 relative error: 1.70e-07
b1 relative error: 4.60e-08
b2 relative error: 6.89e-09
b3 relative error: 1.82e-10
```

As another sanity check, make sure you can overfit a small dataset of 50 images. First we will try a three-layer network with 100 units in each hidden layer. You will need to tweak the learning rate and initialization scale, but you should be able to overfit and achieve 100% training accuracy within 20 epochs.

```
# TODO: Use a three-layer Net to overfit 50 training examples.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
```

```
  'y_val': data['y_val'],
}

##############################################################################
######
##############################################################################
######
learning_rate = 1e-3
weight_scale = 1e-1
##############################################################################
######
#                              END OF YOUR CODE
#
##############################################################################
######

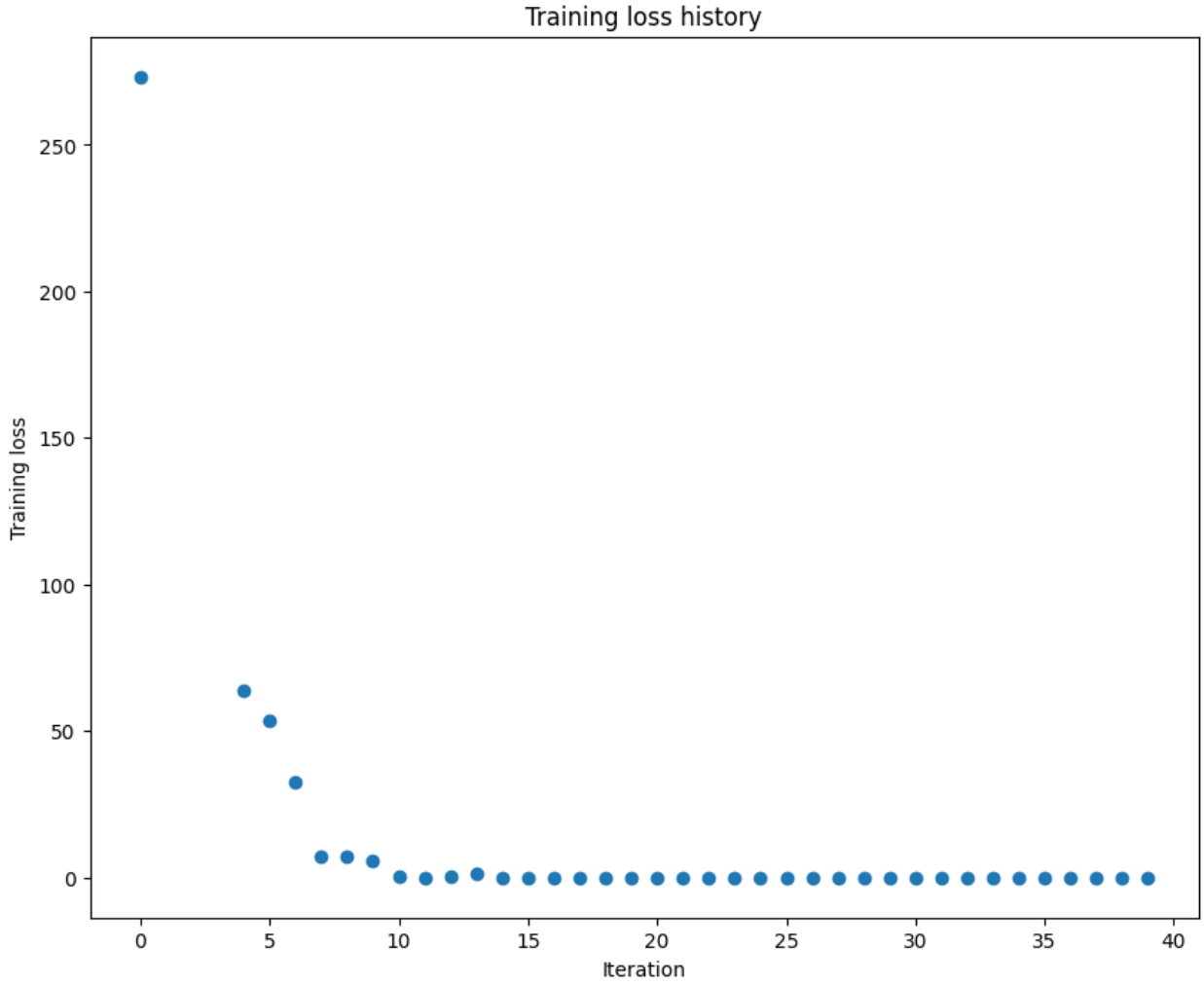
model = FullyConnectedNet([100, 100],
              weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
                print_every=10, num_epochs=20, batch_size=25,
                update_rule='sgd',
                optim_config={
                  'learning_rate': learning_rate,
                }
         )
solver.train()

plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
```

```
(Iteration 1 / 40) loss: 273.298408
(Epoch 0 / 20) train acc: 0.260000; val_acc: 0.112000
(Epoch 1 / 20) train acc: 0.300000; val_acc: 0.121000
(Epoch 2 / 20) train acc: 0.480000; val_acc: 0.128000
(Epoch 3 / 20) train acc: 0.680000; val_acc: 0.121000
(Epoch 4 / 20) train acc: 0.920000; val_acc: 0.127000
(Epoch 5 / 20) train acc: 0.920000; val_acc: 0.137000
(Iteration 11 / 40) loss: 0.534936
(Epoch 6 / 20) train acc: 0.960000; val_acc: 0.137000
(Epoch 7 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 8 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 9 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 10 / 20) train acc: 1.000000; val_acc: 0.132000
(Iteration 21 / 40) loss: 0.000047
```

```
/home/wj/ucb/cs182/hw0code/deeplearning/layers.py:166: RuntimeWarning:
divide by zero encountered in log
  loss = -np.sum(np.log(probs[np.arange(N), y])) / N

(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.132000
(Iteration 31 / 40) loss: 0.000054
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.132000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.132000
```



Now try to use a five-layer network with 100 units on each layer to overfit 50 training examples. Again you will have to adjust the learning rate and weight initialization, but you should be able to achieve 100% training accuracy within 20 epochs.

```
## TODO: Use a five-layer Net to overfit 50 training examples.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

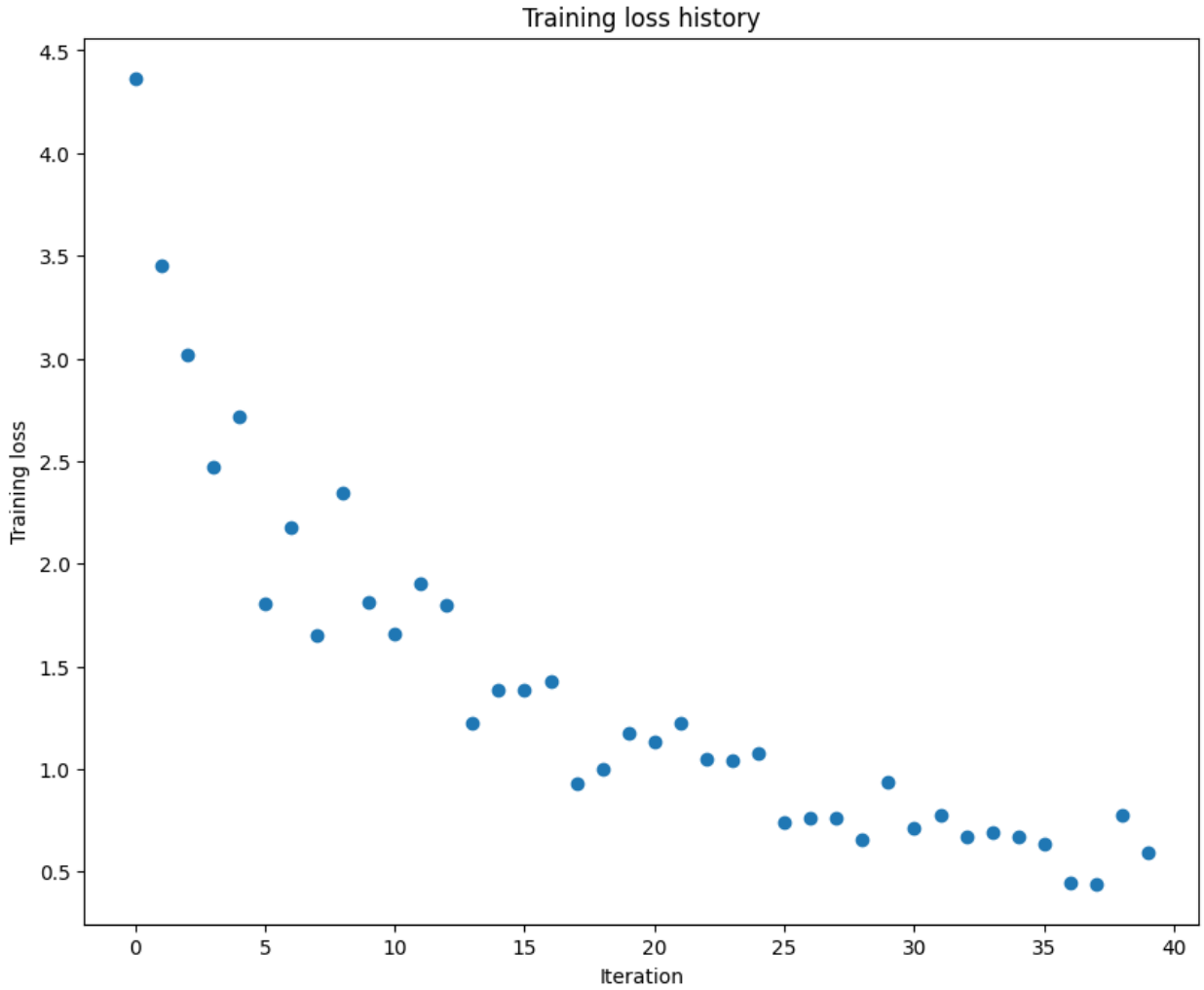
################################################################################
######
# TODO: Tune these parameters to get 100% train accuracy within 20
epochs. #
################################################################################
######
learning_rate = 1e-3
weight_scale = 5e-2
################################################################################
######
#                              END OF YOUR CODE
#
################################################################################
######

model = FullyConnectedNet([100, 100, 100, 100],
                weight_scale=weight_scale, dtype=np.float64)
solver = Solver(model, small_data,
                print_every=10, num_epochs=20, batch_size=25,
                update_rule='sgd',
                optim_config={
                  'learning_rate': learning_rate,
                }
         )
solver.train()

plt.plot(solver.loss_history, 'o')
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.show()
```

```
(Iteration 1 / 40) loss: 4.365196
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.094000
(Epoch 1 / 20) train acc: 0.160000; val_acc: 0.111000
(Epoch 2 / 20) train acc: 0.280000; val_acc: 0.115000
(Epoch 3 / 20) train acc: 0.320000; val_acc: 0.121000
(Epoch 4 / 20) train acc: 0.380000; val_acc: 0.133000
(Epoch 5 / 20) train acc: 0.500000; val_acc: 0.133000
(Iteration 11 / 40) loss: 1.654317
```

```
(Epoch 6 / 20) train acc: 0.540000; val_acc: 0.128000
(Epoch 7 / 20) train acc: 0.580000; val_acc: 0.139000
(Epoch 8 / 20) train acc: 0.700000; val_acc: 0.142000
(Epoch 9 / 20) train acc: 0.700000; val_acc: 0.157000
(Epoch 10 / 20) train acc: 0.760000; val_acc: 0.147000
(Iteration 21 / 40) loss: 1.130693
(Epoch 11 / 20) train acc: 0.740000; val_acc: 0.143000
(Epoch 12 / 20) train acc: 0.740000; val_acc: 0.148000
(Epoch 13 / 20) train acc: 0.840000; val_acc: 0.151000
(Epoch 14 / 20) train acc: 0.840000; val_acc: 0.152000
(Epoch 15 / 20) train acc: 0.900000; val_acc: 0.144000
(Iteration 31 / 40) loss: 0.710055
(Epoch 16 / 20) train acc: 0.900000; val_acc: 0.146000
(Epoch 17 / 20) train acc: 0.880000; val_acc: 0.143000
(Epoch 18 / 20) train acc: 0.900000; val_acc: 0.141000
(Epoch 19 / 20) train acc: 0.900000; val_acc: 0.143000
(Epoch 20 / 20) train acc: 0.960000; val_acc: 0.151000
```



Question:

Did you notice anything about the comparative difficulty of training the three-layer net vs training the five layer net?

Please include your response in the written assignment submission.

Answer:

Based on the results from training both the three-layer and five-layer networks on a small dataset of 50 examples, I observed the following key differences in training difficulty:

1. **Training Accuracy**: The three-layer network achieved 100% training accuracy within 20 epochs, while the five-layer network reached only 96% training accuracy in the same number of epochs. This indicates that the three-layer network was able to overfit the small dataset more easily and completely.

2. **Loss Convergence**: The three-layer network's loss decreased rapidly and reached very low values (close to zero), while the five-layer network's loss decreased more slowly and remained higher throughout training. The three-layer network showed a smoother and faster convergence.

3. **Optimization Challenges**: The five-layer network appeared to be more difficult to optimize, likely due to increased model complexity and depth-related issues such as vanishing gradients, which can make it harder for gradients to propagate effectively through more layers. This required more careful tuning of hyperparameters (like weight scale) to achieve good performance.

In summary, the five-layer network was comparatively more difficult to train than the three-layer network, as it did not achieve perfect overfitting on the small dataset within the same number of epochs and exhibited slower convergence. This highlights the increased optimization challenges that come with deeper network architectures.